ISOVALENT
now part of cisco

Isovalent Networking for Kubernetes: Design Guide for Deployment with Cisco ACI and NX-OS

EARLY RELEASE

Authors:

Camillo Rossi, Technical Leader – Cisco
Rob van der Kind, Technical Leader - Cisco

June 2025

Design Guide

Isovalent and Cisco DC Fabric Design Document

Camillo Rossi, Rob van der Kind, 9-June-2025

Table of Contents

Introduction **5**
Executive Summary 5
Document Structure 7
Kubernetes 8
Isovalent Networking for Kubernetes (Cilium) 8
Isovalent 9
How to Request Design Assistance 9
Contact Information 9
Goals 9
Designs **10**
ACI Basic Design 10
ACI Advanced Design with Maglev for ECMP Flow Consistency (Recommended) 11
EVPN/VXLAN NX-OS Fabric Design 12
Common Features 12
Native Routing and Auto Direct Node Routes 12
Isovalent Networking for Kubernetes BGP Control Plane 13
Cilium Egress Gateway 13
XDP Acceleration 13
ACI Advanced Design and EVPN/VXLAN Design Only Features 13
Maglev 13
Direct Server Return 14
ACI Basic Design **15**
Cluster EPG Physical Connectivity 16
Selective BGP Peering for Service Advertisement 16
Cilium BGP Design Considerations 17
Cilium Egress Design Considerations 18
Egress Gateway and ACI Integration 18
Design Trade-offs 19
ACI Advanced Design **20**
Design Trade-offs 20
Cluster L3Out Physical Connectivity 20

© 2025 Cisco and/or its affiliates. All rights reserved.

BGP Design 21
ECMP Considerations 21
Cilium Egress Design Options 22
Option 1: Egress IP Advertisement via BGP (Preferred) 22
Option 2: Egress IP Classification with ESGs 22
Design Trade-offs 24

ACI BGP Design 24
L3Out Physical Connectivity 24
BGP Dynamic Neighbors 25
BGP Peering with ACI Prior to 6.1(2) 25
Next-Hop Propagation and Ignore IGP Metric - ACI 6.1(2)+ Required 25
Max BGP ECMP Path (Optional) 26
BGP Graceful Restart 26
BFD with Cilium BGP 26
BGP Hardening (Optional) 26

ACI Scale Testing 27
Generic ACI Scale Considerations 27
Conducted Tests: 28
Adding/Removing BGP Peers 28
Reloading a Kubernetes Node 28
Cilium Agent Upgrade/Restart 28
Reloading an ACI Anchor Node 28
Example configuration - Advanced Design With OpenShift 29
ACI configuration 29

Nexus VXLAN-EVPN Design with Cilium 31
Cluster Physical Connectivity 32
BGP Design 32
BGP Dynamic Neighbors 33
BGP Graceful Restart 33
BFD With Isovalent Networking for Kubernetes 33
BGP hardening (optional) 33
ECMP Considerations 33
Isovalent Networking for Kubernetes Egress Design 35
Egress IP Advertisement Over BGP (Preferred Option) 36
Securing Ingress and Egress Traffic 36
Design Trade-offs 37
Closing Remarks 37

© 2025 Cisco and/or its affiliates. All rights reserved.

Authors

Camillo Rossi

Technical Leader – Cisco

Camillo has nearly 20 years of experience in the datacenter space, starting in technical support before transitioning to datacenter design and engineering. Currently, he specializes in ACI and NX-OS fabrics, driving integration with Kubernetes and working on Cisco DC Day 2 Operations solution. Camillo is passionate about bridging modern cloud-native technologies with robust datacenter infrastructure.

Rob van der Kind

Technical Leader - Cisco

Rob is a Technical Leader in Cisco's Datacenter Networking group, with over a decade of experience in datacenter technologies. He specializes in the programmability and automation of Cisco's ACI and NX-OS fabrics, with a strong focus on integrating Kubernetes and cloud-native technologies. Rob works on open-source and automation efforts to help customers integrate modern technologies into reliable, scalable datacenter environments.

© 2025 Cisco and/or its affiliates. All rights reserved.

Introduction

This Best Practices for the Modern Datacenter documentation covers Kubernetes deployments with Cisco ACI and Isovalent Networking for Kubernetes. This document provides guidance on networking best practices for deploying Kubernetes Clusters using Isovalent Networking for Kubernetes (based on Cilium) as the CNI. While it aims to cover a wide range of scenarios and design considerations, please note that not all options are covered. Alternative designs exist, and users are encouraged to conduct their own research and adjust designs to meet specific needs and requirements.

This Best Practices document serves as a guide to help navigate various concepts and options to find a balance between features and complexity when deploying Kubernetes Clusters in the Datacenter. The goal is to provide users with the knowledge and tools needed to implement effective and efficient solutions.

Executive Summary

Application modernization is a growing trend, with many organizations refactoring or re-architecting existing applications to leverage cloud-native technologies for improved agility and scalability. However, a significant portion of applications remain in their legacy state due to various factors like complexity, cost, and risk aversion, resulting in a hybrid environment where both modernized and traditional applications coexist.

To effectively manage traffic between modernized, containerized workloads and legacy applications, a solution that bridges the gap between these environments is essential. Cisco Datacenter (DC) Fabrics (ACI or NX-OS) can provide network policy and connectivity for the legacy workloads, while Cilium manages the network policies within the Kubernetes clusters hosting the modernized applications. By leveraging the designs provided for the Cisco DC Fabrics and Cilium, organizations can achieve consistent security and observability across both their legacy and cloud-native environments, ensuring seamless communication and control.

Within Kubernetes, Cilium excels at enforcing east-west network policies, controlling the communication between microservices and pods within the cluster. Leveraging eBPF technology, Cilium provides efficient and granular control over this internal traffic, allowing administrators to define policies that restrict which services can communicate with each other based on identity, labels, or other criteria. This east-west policy enforcement is crucial for securing Kubernetes environments, limiting the blast radius of potential security breaches, and ensuring compliance with regulatory requirements. Additionally, Cilium enhances observability by offering real-time insights into network traffic, enabling administrators to monitor and troubleshoot network behavior effectively.

The designs provided on document strike a balance between robust security policies, service availability, and the need for seamless and optimal communication across diverse application architectures.

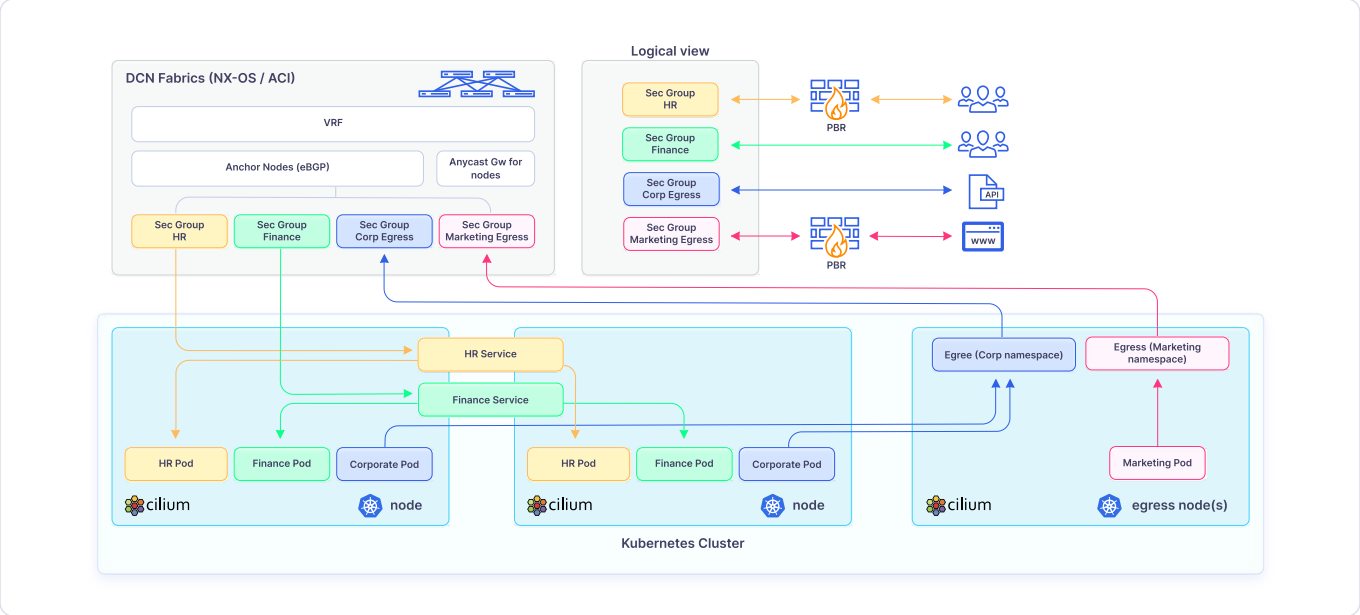
© 2025 Cisco and/or its affiliates. All rights reserved.

At a high level, the following traffic patterns are considered:

- **Node-to-node Communication.** This forms the foundation for much of the networking functionality within a Kubernetes cluster, enabling pods, services, and control plane components to interact effectively. Cilium Native Routing is a networking mode in Cilium where the network packets are directly routed between nodes using the Linux kernel's native routing capabilities, instead of relying on overlay networks like VXLAN. This still allows for all the benefits provided by eBPF while removing the encapsulation overhead, and offering improved performance.

- **External Service Access.** In Kubernetes, a Service is an abstraction that defines a logical set of Pods and a policy by which to access them. It provides a stable IP address for accessing applications running in Pods, regardless of how those Pods are scaled or rescheduled. The IPs allocated to these services are deterministic. Each service is advertised to the Cisco DC Fabrics via BGP as a /32 host route, which can easily be classified using external EPGs (ExtEPGs) or Group Policy Option (GPO). This allows the user to apply security policies that control communication to the service, and where needed, apply service redirection to for example a Firewall. BGP is a well-established and highly scalable routing protocol. Advertising Kubernetes services via BGP allows you to easily scale your services and adapt to changing network conditions.

- **Egress Communication,** also known as Pod-initiated traffic. Pod-initiated traffic encompasses any network connection originating from a Kubernetes pod to external services, APIs, databases, or other resources. Pods in Kubernetes are ephemeral, meaning they can be created, destroyed, and rescheduled frequently. As a result, Pods receive dynamically assigned IP addresses. Relying on these IPs for security policies or auditing is unreliable because they change. Kubernetes typically uses Source Network Address Translation (SNAT) on the worker nodes to allow Pods to access external networks. SNAT replaces the Pod's IP address with the node's IP address. This means that external services only see the node's IP, losing the original identity and context of the Pod that initiated the traffic. Egress Gateways in Kubernetes provide controlled exit points for traffic leaving the cluster. High Availability (HA) for these egress gateways further enhances the reliability and resilience of this egress. Similar to Service IP allocation, allocating IP addresses to Egress Gateways is deterministic. A user can leverage Labels in Kubernetes to specify how Pods, or even entire Namespaces egress the cluster. By combining the deterministic characteristics of Egress Gateway and External EPGs or ESGs with IP Selectors, the user can apply granular control to which services Pods have access.



High level design overview

© 2025 Cisco and/or its affiliates. All rights reserved.

The illustration offers a simplified view of how policy can be applied to control north/south communication to and from a Kubernetes cluster. In this example, the HR users are able to access the HR application - but are first redirected to a firewall for inspection. The users of the Finance app do not have to pass through the firewall, and can only access the Finance services. Pods in the Corporate Namespace are able to access other applications within the Datacenter, whereas Pods in the Marketing namespace only have access to the internet via a Firewall.

To summarize, the combination of Cisco DC Fabrics and Cilium offers a unified solution for managing hybrid environments, bridging legacy and cloud-native applications:

- **Security and Observability:** The Cisco DC Fabrics provides consistent security policies for legacy applications, while Cilium enforces granular network policies within Kubernetes clusters.
- **Traffic Control:** Cilium handles east-west traffic, optimizing internal communications, while ACI excels in north/south traffic management, ensuring precise control and the ability to insert services.
- **Service and Egress Management:** Deterministic external service access via BGP and controlled egress communication using Egress Gateways and ESGs enhance scalability and security.

Document Structure

This design document includes the following chapters/sections:

- **Introduction**
- **Common features:** This chapter describes the Cilium features that are design agnostic.
- **Advanced features:** This chapter describes features only used in the ACI advanced as well as the VXLAN/EVPN design options.

- **ACI Designs:**
  - **Basic Design:** The most straightforward design to help users get started, ideal for testing basic functionalities.
  - **Advanced Design:** An optimized, high-scale design that trades some configuration simplicity for increased scalability. (Recommended)
  - **ACI BGP Design:** This section provides details on the ACI BGP configuration, which is consistent for both the Basic and Advanced designs.
  - **Scale Testing:** A collection of scale limits and tests performed for the Advanced design.
  - **Config Examples:** Configuration examples for ACI.

- **NX-OS Designs:**
  - **Nexus VXLAN-EVPN design with Cilium:** An optimized, high-scale design that trades some configuration simplicity for increased scalability. (Recommended).

© 2025 Cisco and/or its affiliates. All rights reserved.

Kubernetes

Kubernetes has become the de facto standard for container orchestration in today's cloud-native ecosystem, providing a robust framework for deploying, scaling, and managing containerized applications. As enterprises adopt Kubernetes, they often need to ensure smooth network connectivity and service discovery across diverse and dynamic environments. Traditionally, implementing BGP peering to all nodes in a Kubernetes cluster has been a common strategy to provide security and visibility into the workload running on the cluster, as well as handling external services routing and load balancing requirements.

BGP peering enables Kubernetes nodes to directly exchange routing information, improving network traffic flow and reducing latency. However, when designing BGP peering for Kubernetes clusters, particularly as they grow in size and complexity, several key considerations help ensure efficient, scalable, and manageable network configurations.

Three design solutions are proposed:

- A basic design for ACI fabrics trading some features and performance for ease of configuration and operations.
- A slightly more advanced design for ACI fabrics that trades some simplicity for better performance and resilient ECMP hashing towards services.
- A design for EVPN/VXLAN NX-OS fabrics

At present, these designs do not require additional components from Cisco or other third-party vendors to meet their objectives. Users are encouraged to deploy programmatically or even automate the configuration on ACI, Nexus Dashboard or NX-OS. These efforts can aid in repeatability, reduce risk, and accelerate deployments. However, such efforts are optional and are beyond the scope of this design document.

This document will examine the technical foundations, implementation considerations, and operational benefits of this approach. It guides readers through the process of implementing this combined model, ensuring that network reliability, performance, and security are maintained and significantly improved.

Isovalent Networking for Kubernetes (Cilium)

Isovalent Networking for Kubernetes provides networking, security, and observability capabilities for containerized applications, particularly in Kubernetes environments. It uses eBPF (extended Berkeley Packet Filter), a powerful Linux kernel technology, to manage network policies, service connectivity, and security efficiently at the application layer. It is the enterprise offering of the Cilium open-source project, created by Isovalent.

Key features of Isovalent Networking for Kubernetes include:

Advanced Network Security: Enables fine-grained security policies defined based on application-layer identities rather than just IP addresses. This allows for more precise control over communication between microservices.

Scalability and Performance: By using eBPF, Isovalent Networking for Kubernetes achieves high performance and scalability, running network policies directly in the Linux kernel without needing complex user-space processing.

Service Mesh Integration: Can be integrated with service mesh architectures to provide transparent, high-performance networking and security features that complement service mesh features.

Observability: Offers robust observability tools, allowing users to monitor and troubleshoot network traffic and security policies with detailed metrics and visibility into application communication patterns.

Kubernetes Native: Designed for Kubernetes, it seamlessly integrates to provide native support for network policies and service discovery.

Isovalent Networking for Kubernetes is widely adopted in cloud-native environments for its ability to enhance security and network performance while providing comprehensive observability and integration with existing Kubernetes infrastructure.

© 2025 Cisco and/or its affiliates. All rights reserved.

Isovalent

As the creators of eBPF, Cilium, and Tetragon, Isovalent is a key part of innovation in cloud-native ecosystems. All major cloud providers have adopted Isovalent's technologies, making it the de-facto standard for cloud-native networking and security.

The Isovalent Enterprise Platform provides advanced Networking, Security, and Observability solutions for Kubernetes and cloud-native environments. This is combined with a global customer support offering.

Isovalent joined Cisco via acquisition in April 2024.

The Isovalent Platform is used by enterprises such as Goldman Sachs, J.P. Morgan Chase, UBS, and S&P Global, as well as technology leaders like Databricks, Adobe, and Confluent, and leading AI Large Language Model (LLM) providers.

Disclaimer: This document has been developed specifically for the **Isovalent Networking for Kubernetes** version. No support or guidance is provided for the Cilium open-source version. Cisco TAC does not provide support for Cilium, and Isovalent Networking for Kubernetes support can be purchased separately.

How to Request Design Assistance

It is understood that every network environment is unique. Assistance is available to help adjust solutions to meet specific needs. For design assistance, experts are available to help through the process. Contact information:

Contact Information

- Contact Isovalent
- Contact the Cisco account manager (Global Contact Page)
- Request a demo

Support aims to provide personalized guidance to ensure the network design aligns with operational requirements and future growth plans, helping to achieve project goals.

Goals

The design goals for this Kubernetes networking solution focus on addressing key challenges administrators face when managing traffic flow and ensuring secure connectivity within and outside the Kubernetes cluster. The approach implements a robust and scalable load-balancing strategy combined with a secure networking model that connects containerized applications and external non-containerized systems.

Note: With the designs discussed in this document, the role of the network fabric is to provide **North-South** traffic filtering. The network fabric can selectively permit or deny traffic ingressing or egressing the Kubernetes or Openshift cluster using a combination of contracts and security groups. The network fabric **cannot** control intra-cluster (East-West) traffic using the approaches proposed here. This role belongs to Cilium's rich security feature set. Isovalent Networking for Kubernetes includes a powerful yet easy to use GUI tool that lets security administrators create, test, validate and monitor intra-cluster security policies. Using the security model offered by ACI and EVPN/VXLAN fabrics and Cilium together provides a very complementary layered approach to securing the datacenter.

© 2025 Cisco and/or its affiliates. All rights reserved.

Designs

This design document outlines two approaches to achieving the networking objectives. Each option is designed for different priorities and operational considerations.

Regardless of the chosen option, **all** designs provide the following outcomes:

- **High Performance**
  - By using Native Routing it is possible to increase network stack performance by removing the Node-to-Node overlay as well as enabling advanced features like BigTCP.
  - **Note:** BigTCP is currently in beta and should not be used for production workloads. However, this design is ready for its future stable release, allowing it to be enabled easily with a simple configuration flag once it becomes available.
- **Efficient load balancing:**
  - Load-balancing that effectively distributes external client traffic to services within the Kubernetes cluster.
  - Utilize cluster nodes to perform BGP peering, enabling Layer-3/Layer-4 Equal-Cost Multi-Path (ECMP)-based load balancing.
- **External service security:**
  - The load balancer Ips are mapped to Security Groups to help enforce security policies through contracts.
- **Pod identity**
  - With the Egress Gateway feature, it's possible to masquerade Pod traffic leaving the cluster with predictable IPs associated with the gateway nodes. As an example, this feature can be used in combination with legacy firewalls to allow traffic to legacy infrastructure only from specific pods within a given namespace.

Note: To maintain clarity and ease of reading, all configuration snippets and examples have been grouped into dedicated example sections within this document. This structure ensures that relevant information and guidance on configurations are easily accessible and presented in a clear manner. Readers can refer to these sections for detailed configurations and examples.

ACI Basic Design

This design serves as an excellent starting point for quickly getting started, allowing exploration of Cilium's capabilities easily.

- **Objectives:**
  - Ability to run on any ACI Software version.
  - Provide a straightforward and easily maintainable solution.
  - Utilize ACI for External Service load-balancing.
  - Utilize ACI Contracts to control North-South traffic.
- **Approach:**
  - All Cluster Nodes are deployed in an EPG and linked to an ESG.
  - Native routing mode: Since all nodes share the same L2 Domain (the Node BD), Pod-to-Pod traffic can be directly routed.
  - Dedicated Egress nodes, with the Egress IP(s) linked to an ESG.
  - Dedicated Ingress nodes peering with BGP for External Services.

© 2025 Cisco and/or its affiliates. All rights reserved.

- **Benefits:** Quick deployment, reduced complexity, and lower operational overhead.
- **Limitations:**
  - Cilium selects service backends randomly and ensures that traffic remains sticky to the backend. However, this method can cause issues if a node fails or if there is a change in the ECMP Next Hops. This can result in traffic being sent to a different Kubernetes node, which lacks context about which backend was handling the connection. This can lead to unexpected disruptions on connection-oriented protocols like TCP, as client connections get reset by the newly selected backends.
  - May not fully optimize network throughput in scenarios with complex traffic patterns.

ACI Advanced Design with Maglev for ECMP Flow Consistency (Recommended)

This design offers a more comprehensive solution, ideal for production environments. It includes advanced features and optimizations recommended for using the full potential of Cilium in a stable and scalable manner. While it may require more initial setup and configuration, this design ensures robust performance and resilience, making it well-suited for enterprise-level deployments.

Objectives:

- Maximize network performance and ensure consistent flow distribution in ECMP scenarios with Maglev.
- Utilize ACI Contracts to control North-South traffic.

Approach:

- All Nodes are deployed behind a L3Out.
  - Use a `Node` External EPG to secure Kubernetes Node traffic.
  - Use one External EPG per exposed External Service for North-South traffic accessing Kubernetes services. Note that external services can also be grouped in one or multiple EPGs.
  - Use Maglev hashing to maintain ECMP flow consistency. It is suited for environments needing optimal load distribution and performance. Maglev requires the following:
    - Direct Server Return: the response is sent directly back to the client from the pod, skipping the original node that received the request.
    - Native routing mode: The native routing mode uses the routing capabilities of the network Cilium runs on instead of performing encapsulation for Pod-to-Pod traffic.
- Dedicated Egress nodes, with the Egress IP(s) linked to an ESG.

- **Benefits:** Improved flow consistency, enhanced network utilization, and better handling of dynamic traffic patterns.
- **Limitations:**
  - ACI 6.1(2) or later is required.
  - Node IP visibility is reduced as they are placed behind an L3Out.

© 2025 Cisco and/or its affiliates. All rights reserved.

EVPN/VXLAN NX-OS Fabric Design

Unlike ACI, which presented two distinct approaches (Basic vs. Advanced), the NX-OS section presents a single, highly scalable, consistent approach using a centralized routing design, and leverages Security Groups and Contracts (GPO) for North-South traffic control. This design is ideal for production environments.

- **Objectives:**
  - Maximize network performance and ensure consistent flow distribution in ECMP scenarios with Maglev.
  - Utilize Contracts to control North-South traffic with GPO.
- **Approach:**
  - Centralized routing design whereby a pair of anchor nodes provide an eBGP control plane
  - An anycast gateway providing a L2 segment for node-to-note communication
  - Use a Node External EPG to secure Kubernetes Node traffic.
  - Use Maglev hashing to maintain ECMP flow consistency. It is suited for environments needing optimal load distribution and performance Maglev requires the following:
    - **Direct Server Return:** the response is sent directly back to the client from the pod, skipping the original node that received the request.
    - **Native routing mode:** the native routing mode uses the routing capabilities of the network Cilium runs on instead of performing encapsulation for Pod-to-Pod traffic.
  - Use of Security Groups and contracts to control North-South traffic accessing Kubernetes services.
  - Use of Security Groups and contracts to control North-South traffic for pod-initiated traffic through EgressGateway.
- **Benefits:** Improved security, flow consistency, enhanced network utilization, and better handling of dynamic traffic patterns.

Common Features

Native Routing and Auto Direct Node Routes

Using these two features allows highly efficient packet forwarding between pods without the need for additional encapsulation or overlay networks.

In a Kubernetes deployment running in direct mode on a network fabric, each pod is assigned an IP address from a routable subnet with direct Layer 2 or Layer 3 connectivity. This allows pods to communicate natively over the physical network infrastructure, eliminating the latency, CPU overhead, and complexity typically associated with tunneling protocols such as VXLAN or GRE.

By maintaining direct routing to each pod subnet, the solution ensures minimal packet traversal path and maximum throughput delivering the best possible performance for East-West traffic in the cluster. This architecture is especially beneficial in performance-sensitive environments where low latency and deterministic network behavior are critical.

Warning: This design aims to cover the vast majority of customer needs. However, if a cluster grows beyond 1,000 nodes, changes to the design might be needed. In these cases, it is strongly suggested to contact Isovalent for more help and support.

© 2025 Cisco and/or its affiliates. All rights reserved.

Isovalent Networking for Kubernetes BGP Control Plane

The Cilium BGP Control Plane Enterprise feature lets Cilium announce routes to connected routers using the Border Gateway Protocol (BGP). It makes pod networks or services of Type LoadBalancer reachable from outside the cluster in environments that support BGP. In Cilium, the BGP Control Plane doesn't set up the Linux host's data path directly, so it cannot be used by itself to create IP connections within the cluster or to external IPs.

Cilium Egress Gateway

The Egress Gateway feature allows sending traffic that starts from pods and goes to specific network ranges (CIDRs) outside the cluster through certain chosen nodes.

When the egress gateway feature is turned on and egress gateway policies are active, packets leaving the cluster have their source IP address changed to selected, known IPs `(egressIP)` linked to the gateway nodes. This lets network administrators set network rules for pods or namespaces making outbound connections.

Warning: Only Isovalent Networking for Kubernetes supports Egress Gateway High Availability (making sure the egress path keeps working even if one gateway node fails).

XDP Acceleration

XDP Acceleration helps speed up NodePort, LoadBalancer services, and services with externalIPs when incoming requests need to be forwarded to a backend pod located on a different node. This feature was added in Cilium version 1.8. It works at the XDP (eXpress Data Path) layer, where eBPF runs directly in the network card driver instead of higher up in the system.

Most network drivers that support 10Gbps speeds or higher also support native XDP if using a recent kernel version. For a list of drivers that support XDP, please refer to the official Cilium documentation.

ACI Advanced Design and EVPN/VXLAN Design Only Features

Maglev

Using advanced load balancing methods in Kubernetes clusters is important for keeping performance high and using resources well. Combining Cilium with Maglev hashing offers a strong solution for stable and efficient load spreading, especially when using Equal-Cost Multi-Path (ECMP) routing.

Maglev consistent hashing reduces problems by making sure each load balancing node has the same view and order for its list of backend servers. This means choosing the backend server based on the packet's details (5-tuple hash) will always send the traffic to the same server without needing to share information with other nodes. This not only makes the system more reliable if failures happen but also spreads the load better because new nodes added to the cluster will choose the same backend server consistently.

© 2025 Cisco and/or its affiliates. All rights reserved.

Maglev consistent hashing

Besides that, the Maglev consistent hashing method ensures load is spread evenly among backend servers and causes few problems when backends are added or removed. Specifically, a connection is very likely to pick the same backend server after a server is added or removed as it did before. When a server is removed, the backend lists are updated with few changes for other servers. Usually, this means at most 1% of connections might go to a different server than before.

This is especially important for networks that don't support ECMP Resilient Hashing.

To use Maglev hashing successfully in Kubernetes with Cilium, a main requirement is that Kubernetes nodes can target the Pod IP directly when routing traffic. This ability is needed for keeping load balancing consistent and efficient. Also, for best performance and less delay, the node where the Pod is located should be able to send replies directly back to the network. This direct reply method needs Direct Server Return (DSR).

This blog post explains Maglev in more detail.

Direct Server Return

When Direct Server Return (DSR) mode is turned on in Cilium, handling traffic from outside the nodes is made much better. In DSR mode, after a request reaches a backend pod on a different node, the reply is sent directly back to the client from that pod. It bypasses the original node that first received the request. This removes the extra step needed for changing the source IP back (reverse SNAT), reducing delay and improving overall network performance.

DSR mode offers two main benefits:

- **Keeps Source IP:** Because the backend pod sends the reply directly to the client, the client's original source IP is kept. This is very helpful for security and monitoring. It lets network rules and logs on the backend node correctly see and match the client's real IP address. This can be vital for setting detailed security rules and for fixing problems.
- **Less Delay and Load:** By removing the need for the reply to go back through the node that first received the request, delay is reduced. This also lowers the processing work on that initial node, as it doesn't have to handle changing IPs back for replies. This allows it to handle incoming requests more efficiently.

Overall, editing DSR in Cilium improves network traffic flow, boosts security, and uses resources better within the Kubernetes cluster. It's a good choice for setups where performance and knowing the client's true IP are important.

Note: To make sure Maglev and Direct Server Return (DSR) work correctly, the service traffic policy needs to be set to `externalTrafficPolicy: Cluster`. This setup let's all nodes forward traffic to the external service IP, ensuring

© 2025 Cisco and/or its affiliates. All rights reserved.

connections work smoothly. To keep things simple, this design uses BGP peering between all nodes and the fabric. However, if the goal is to reduce the number of BGP neighbors, BGP peering can be set up only with a chosen group of nodes. This can be done using node labels and node selectors in the `IsovalentBGPPeerConfig`, allowing more specific network setups.

If your egress nodes are set up to announce IPs over BGP, it is important to ensure they also have BGP connections with the fabric. To keep egress and ingress nodes separate, multiple `IsovalentBGPPeerConfig` settings can be used. This method lets you manage BGP settings separately for different node roles, improving network separation and operational control.

ACI Basic Design

Note: This design is primarily intended for testing or lab environments. For full production deployments, reviewing and potentially adopting the Advanced Design is recommended.

The basic network infrastructure for this Basic Design is composed of the following required ACI components:

- **Tenant:** The Kubernetes cluster resides within a dedicated or shared ACI tenant.
- **Primary Bridge Domain (BD):** A central BD provides L2 connectivity and L3 gateways for the Kubernetes nodes.
  - Configuration includes a **primary subnet** matching the Kubernetes node IP range. The BD Subnet Virtual IP (SVI) serves as the default gateway for the nodes.
  - Configuration includes **one or more secondary subnets**. These IP ranges are designated for use by the Cilium Egress Gateway feature as source IPs for egress traffic.
- **Primary Endpoint Group (EPG):** An EPG linked to the primary BD is created. The primary network interfaces of all Kubernetes nodes are associated with this EPG (e.g., via static path bindings or VMM integration).
- **Node Endpoint Security Group (ESG):** A node ESG is created. An IP-based selector matching the primary node subnet groups all nodes within this ESG, allowing nodes within the ESG to communicate without contracts.
- **Egress Endpoint Security Groups (ESGs):** Additional `egress` ESGs are created. Each uses an IP-based selector matching specific `Egress IP` addresses (derived from the egress gateway configuration). These ESGs classify egress traffic based on Pod/Namespace identity for granular ACI contract enforcement.
- **Dedicated L3Out for Ingress:** A separate Floating SVI L3Out is configured. This L3Out establishes BGP peering exclusively with a designated subset of Kubernetes nodes (`ingress nodes`) for advertising external Kubernetes Services.

This Basic Design provides the following capabilities:

- Secures traffic initiated *by* Kubernetes nodes (e.g., accessing external services) using ACI contracts applied via the node ESG.

Note: Internal cluster micro-segmentation using CiliumNetworkPolicies is a key feature of Isovalent Enterprise for Cilium itself, but detailed configuration is beyond the scope of this ACI integration guide.

- Secures traffic initiated *from* Pods that is NATted using specific Egress IPs, leveraging ACI contracts applied via the dedicated `egress` ESGs.
- Supports DHCP Relay: Enables Kubernetes nodes to obtain IP addresses automatically during bootstrap via the ACI fabric, simplifying deployment and scaling.
- Provides direct visibility of node IPs within the ACI fabric via the EPG/ESG association.
- Supports heterogeneous node types: The cluster can comprise a mix of bare-metal servers and virtual machines on various hypervisors, provided they connect to the designated ACI EPG.
- Offers routing simplicity: The default gateway for all nodes is the SVI IP address of the primary BD subnet.
- Enables BGP-based ECMP for load balancing external Kubernetes service traffic across the designated `ingress nodes`.

© 2025 Cisco and/or its affiliates. All rights reserved.

Cluster EPG Physical Connectivity

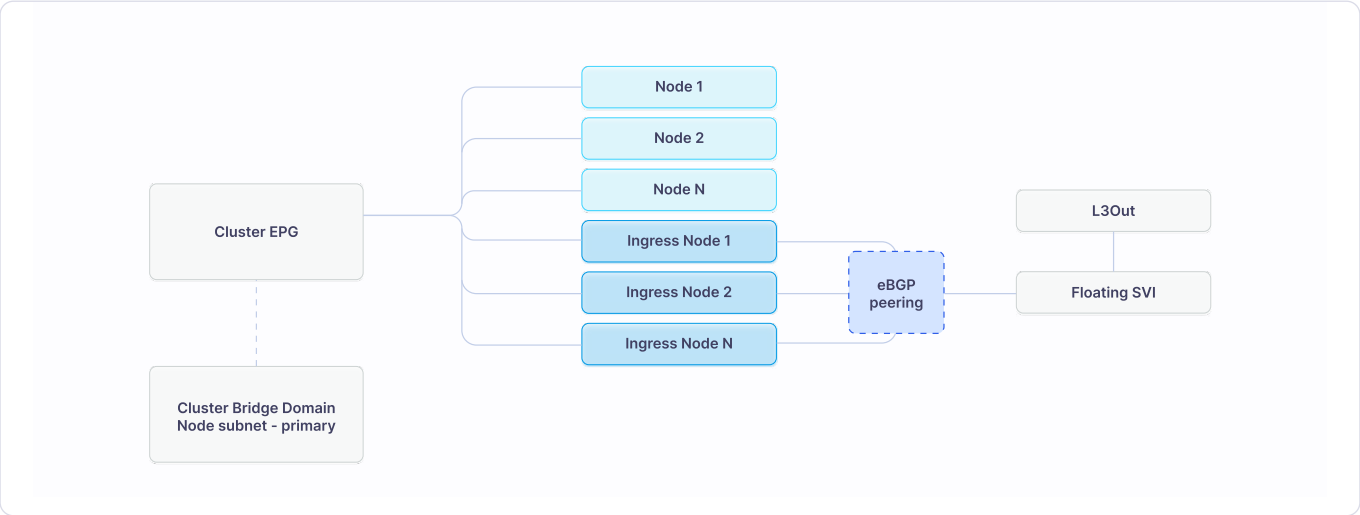
No strict requirements dictate the physical connectivity method for attaching Kubernetes nodes to the cluster EPG, provided the necessary level of redundancy is met. Designs often utilize vPC for host connectivity. Implementing L2 redundancy generally improves failover performance compared to relying solely on L3 convergence mechanisms.

Selective BGP Peering for Service Advertisement

This design employs selective BGP peering. A dedicated L3Out (typically using Floating SVI) is created in ACI solely for establishing BGP sessions with a designated subset of Kubernetes nodes, referred to as `ingress nodes`. These nodes advertise external Kubernetes Service IPs (/32 routes) to the ACI fabric via this L3Out.

The `ingress nodes` require configuration with two logical network interfaces:

1. **Primary Interface:** Connects to the main cluster EPG/BD (same as non-ingress nodes). This handles standard node-to-node communication and uses the BD SVI as its default gateway. This interface's IP should be the primary `node-ip` for kubelet. Keeping `ingress nodes` within the main EPG simplifies node provisioning (e.g., DHCP) and internal cluster communication.
2. **Secondary Interface (for BGP):** Connects (logically or physically) to the network segment defined by the dedicated L3Out. This interface is used for the BGP peering session with the ACI anchor leaves associated with the L3Out.



Selective BGP Peering Design

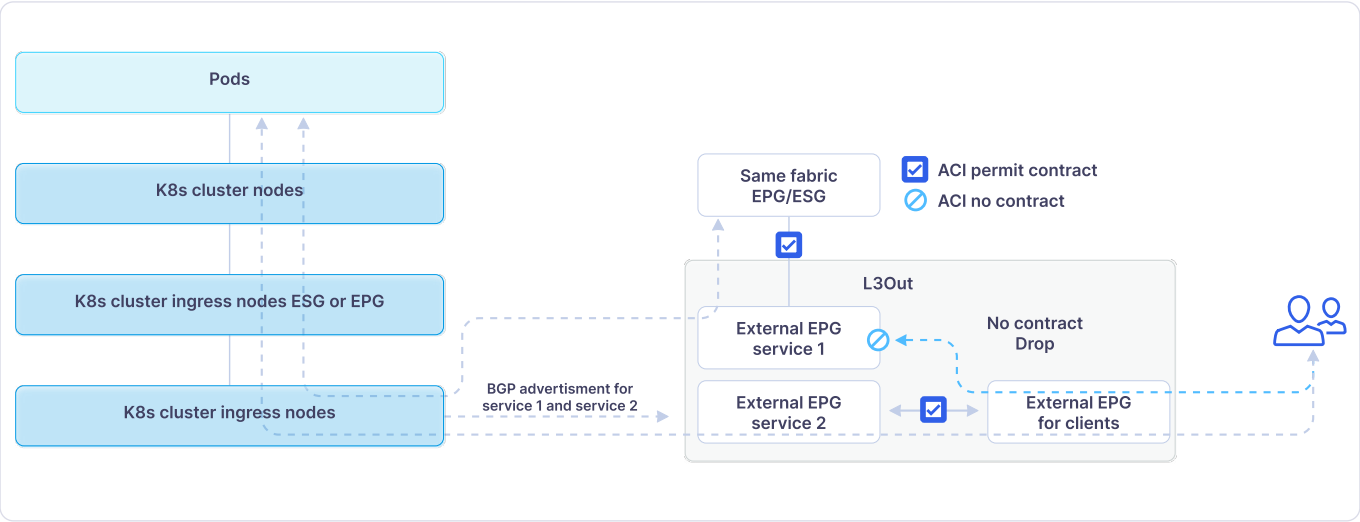
Routing Considerations for Ingress Nodes:

Traffic arriving at an `ingress node` via the L3Out (destined for a Kubernetes Service IP) must have its return traffic sent back out through the same L3Out path to maintain symmetry and avoid potential RPF issues or policy drops on the primary interface path. Linux Policy-Based Routing (PBR) is configured on the `ingress nodes` to achieve this:

- **Create a separate routing table:** (e.g., table ID 100).
- **Add default route in the new table:** In table `100`, configure a default route pointing to the appropriate gateway IP address within the L3Out's network segment (e.g., the Floating SVI IP if used).
- **Create IP routing rules:** Use `ip rule add` commands to direct traffic sourced from the Kubernetes Service IP range(s) handled by this node to use routing table `100`.

This ensures that replies to client requests arriving via the L3Out are routed back out the secondary interface towards the L3Out gateway.

© 2025 Cisco and/or its affiliates. All rights reserved.



Cilium BGP Control Plane Traffic Flows

Note: For nodes configured with multiple network interfaces, it is fundamental to ensure that kubelet's `node-ip` is set correctly on each node. In the Basic Design, this **must** be the IP address of the primary interface connected to the main ACI EPG/BD. Cilium relies on this primary node IP for internal pod-to-pod east-west routing and does not provide mechanisms to choose alternative interfaces for this traffic.

Refer to the ACI BGP Design section for general ACI BGP configuration details applicable here.

Cilium BGP Design Considerations

From the Cilium perspective, the main consideration in this design is determining the number of `ingress nodes` to deploy and deciding whether to dedicate these nodes solely to the ingress function.

Deployment Recommendations:

- **Redundancy:** Deploy a minimum of two `ingress nodes`, connected to a pair of ACI leaf switches handling the L3Out peering. This provides resilience against `ingress node` or leaf failures and allows for hitless maintenance or upgrades.
- **Dedicated vs. Shared:** Depending on cluster scale, ingress traffic volume, and performance requirements, dedicating specific nodes solely for the ingress function can offer advantages:
  - **Performance Predictability:** If regular application Pods are not scheduled onto `ingress nodes`, ingress traffic typically traverses a predictable path (Client -> ACI -> Ingress Node -> Worker Node), leading to more consistent latency.
  - **Resource Isolation:** Dedicated nodes ensure that their CPU, memory, and network bandwidth are fully available for handling ingress traffic and BGP processing, without contention from application workloads.
  - **Reduced Peering Scale:** Since only a subset of nodes establishes BGP peering with the fabric, the overall BGP management overhead on both ACI and Cilium is reduced compared to peering with all nodes.
  - **Specialized Hardware:** Not all nodes in a cluster need identical hardware. High-performance hardware (e.g., faster CPUs, high-throughput NICs) can be specifically utilized for the `ingress nodes`. For example, using CiliumNodeConfig, bare-metal nodes equipped with Mellanox or Intel NICs supporting features like Cilium's Big TCP can be deployed, potentially achieving very high throughput per ingress node.

Refer to the Example configuration section of this document for implementation details.

© 2025 Cisco and/or its affiliates. All rights reserved.

Cilium Egress Design Considerations

For handling egress traffic in this Basic Design, Cilium's Egress Gateway feature utilizes IP addresses sourced from the secondary subnets configured on the ACI BD. These Egress IPs are then classified using ACI ESGs for policy enforcement.

Deployment Recommendations:

The primary consideration is the number of nodes designated to handle egress traffic (egress nodes) and whether these should be dedicated:

- **Redundancy:** Deploy a minimum of two `egress nodes`, physically connected across different ACI leaf switches or pairs for redundancy against node/leaf failures and during maintenance.
- **Dedicated vs. Shared:** Similar to ingress, dedicating nodes for egress can be beneficial depending on scale and requirements, offering resource isolation and potentially specialized hardware advantages. The rationale mirrors the points discussed for dedicated `ingress nodes`.

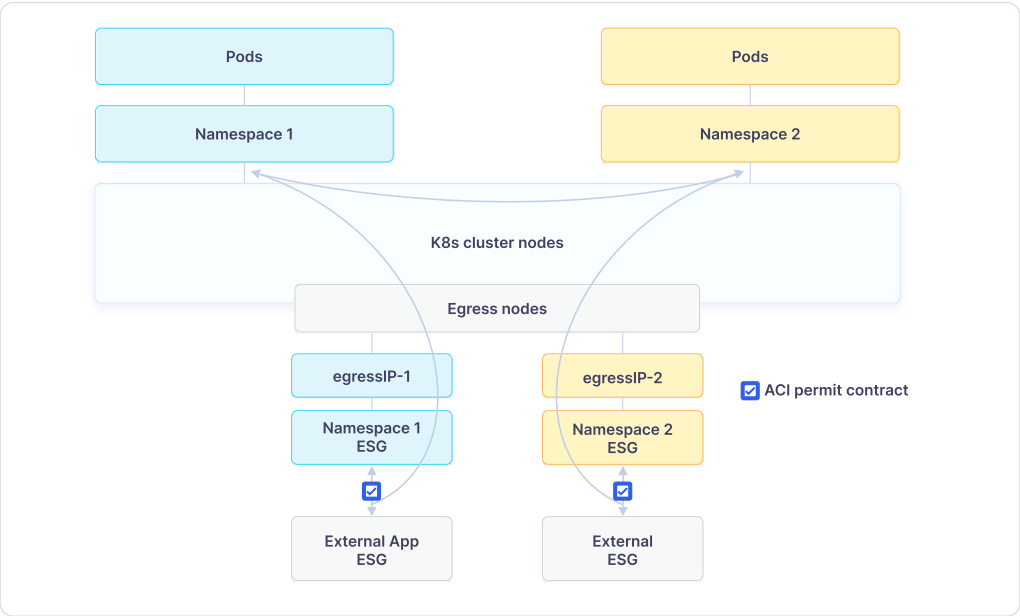
Note: A single node designated as an `egress node` can be assigned multiple Egress IP addresses (drawn from the secondary BD subnets). This allows one physical node to represent multiple distinct egress identities, which can be mapped via Cilium Egress Gateway policies to different Kubernetes Namespaces or Pod selectors. For example, Egress IP-A could be used by Namespace A, while Egress IP-B on the same node serves Namespace B.

Egress Gateway and ACI Integration

In this Basic Design, the integration between Cilium Egress Gateway and ACI policy relies on ESGs applied within the primary cluster BD:

- **Egress IPs from Secondary Subnets:** Cilium Egress Gateway is configured to use IP addresses from the secondary subnet(s) defined on the main ACI BD. These become the fixed source IPs for designated egress traffic.
- **ESG Classification:** ACI ESGs are created with IP-based selectors matching these specific Egress IPs (or ranges).
- **Cilium Policy Mapping:** Standard Cilium Egress Gateway policies map Kubernetes Pods/Namespaces to specific Egress IPs.
- **ACI Contract Enforcement:** Contracts are applied between the egress ESGs (representing specific application egress identities) and external destinations (represented by other ESGs or ExtEPGs), allowing granular control within ACI.

This approach controls traffic leaving the cluster via the Egress Gateway mechanism, while internal cluster traffic remains unaffected by these specific ESG policies.



Egress Gateway Traffic Flow using ESGs

© 2025 Cisco and/or its affiliates. All rights reserved.

Note: While Cilium can advertise Egress Gateway IPs via BGP (as described in the Advanced Design Option 1), this Basic Design favors ESG classification. Using ESGs avoids consuming ExtEPG resources on the L3Out (which might be limited, e.g., ~250 per L3Out instance) for egress classification, preserving those ExtEPGs primarily for classifying external Kubernetes Services advertised via BGP.

Design Trade-offs

This Basic Design aims to provide a relatively straightforward and scalable integration; however, it involves the following trade-offs:

1. `externalTrafficPolicy: Cluster` **Likely Required:** Because incoming traffic for external services first lands on a potentially small subset of `ingress nodes` (which may not host the target backend Pod), using `externalTrafficPolicy: Cluster` is often necessary. This allows the `ingress node` (via kube-proxy or Cilium's service handling) to perform a second layer of load balancing to forward the traffic to a node actually running the backend Pod. This differs from the Advanced design where DSR can optimize the return path.
2. **Potential Ingress/Egress Bottlenecks:** Concentrating all external service traffic (ingress) through the `ingress nodes` and potentially egress traffic through specific `egress nodes` can create bottlenecks if these node pools are not adequately scaled (horizontally or vertically) for the required traffic load.
3. **No Pod IP Advertisement:** This design does not involve advertising Kubernetes Pod IP subnets into the ACI fabric via BGP. Direct reachability to Pod IPs from outside the cluster is generally not a goal here; access is primarily via Kubernetes Services (ingress) or controlled via Egress Gateway (egress).
4. **Ingress Node Complexity:** The requirement for `ingress nodes` to have two interfaces (one in the main EPG/BD, one for the L3Out) and associated Policy-Based Routing adds configuration complexity specific to those nodes.

Trade-off (1) is inherent in designs separating ingress peering from the main node pool without advanced features like DSR fully mitigating the extra hop. Issue (2) is manageable with proper capacity planning and scaling of the ingress/egress node pools. Issue (3) is a fundamental aspect of the design, relying on Services and Egress Gateways for external interaction rather than direct Pod routing. Issue (4) introduces operational overhead for the designated ingress nodes.

© 2025 Cisco and/or its affiliates. All rights reserved.

ACI Advanced Design

The basic network infrastructure for this design is composed of the following core ACI components:

- **Tenant:** The Kubernetes cluster can reside within a dedicated tenant or a shared tenant, particularly in environments hosting multiple clusters.
- **Floating SVI L3Out:** A single L3Out utilizing the Floating SVI feature serves several purposes:
  - Provides the primary L3 gateway and L2 broadcast domain for node-to-node communication for most, if not all, Kubernetes nodes.
  - Facilitates traffic classification and security policy enforcement via ACI Contracts applied to dedicated External EPGs (ExtEPGs) for nodes and services.
  - Establishes BGP peering with all or a subset of Kubernetes nodes for advertising external Kubernetes Services (`/32` routes) and enabling load balancing.
- **Optional Bridge Domain (BD) for Dedicated Egress:** Depending on the chosen egress strategy (detailed later), a separate BD might be configured specifically for nodes handling egress traffic:
  - An EPG within this BD connects the secondary interfaces of designated egress nodes.
  - Multiple Endpoint Security Groups (ESGs) within this EPG can use IP selectors to classify traffic based on specific Kubernetes `Egress IPs`, allowing distinct policies for different Pods or Namespaces.

This design provides the following capabilities:

- Secures traffic entering and leaving the cluster using ACI contracts.
- Supports heterogeneous node types: The cluster can comprise a mix of bare-metal servers and virtual machines running on various hypervisors, provided network connectivity is established.
- Offers routing simplicity: The default gateway for the nodes is the ACI Floating SVI IP address associated with the L3Out.
- Eliminates the need to advertise the Kubernetes Pod CIDR block into ACI.
- Leverages BGP-based ECMP for external Kubernetes service load balancing, potentially enhanced with resilient hashing techniques (like Maglev, implemented by Cilium).
- Achieves near-optimal traffic flows for external services through Direct Server Return (DSR).

Design Trade-offs

Please note that DHCP Relay in L3OUT configurations for clients connected to the L3OUT has several limitations and is currently not recommended for production environments. We are aware of these limitations, and a future ACI release will address and resolve them.

In the meantime, as a workaround, you can use one of the following approaches:

1. Static IP Allocation: Assign static IP addresses to your Kubernetes nodes to eliminate the dependency on DHCP Relay in the L3OUT.
2. DHCP Server or Relay Placement: Place a DHCP server or DHCP relay in the same L3OUT Floating SVI as the Kubernetes nodes

Cluster L3Out Physical Connectivity

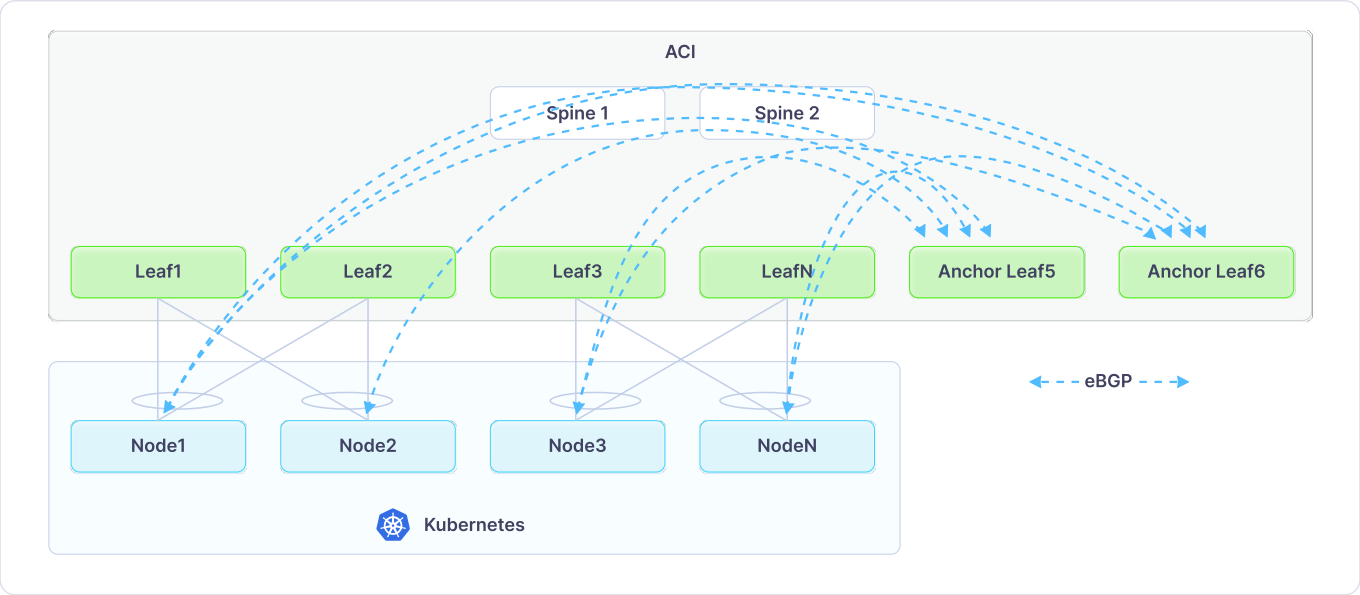
No strict requirement dictates the physical connectivity method for the nodes associated with the L3Out, as long as the desired level of redundancy is achieved. Many deployments will likely utilize vPC for connecting the hosts. Implementing L2 redundancy at the host connection level generally improves failover performance by reducing reliance solely on BGP convergence times.

© 2025 Cisco and/or its affiliates. All rights reserved.

BGP Design

Centralized BGP Peering for Service Advertisement

In this model, all Kubernetes nodes participating in BGP peering establish sessions with a designated pair of ACI leaf switches, often referred to as "anchor" leaves. This peering occurs regardless of the nodes' physical connectivity points within the fabric. This approach simplifies both the physical network configuration and the Cilium BGP setup. As of ACI release 6.1(2) (current as of May 2, 2025), ACI supports up to 2,000 BGP neighbors per leaf switch, a limit unlikely to pose a constraint for a single large Kubernetes cluster. When multiple Kubernetes clusters are connected to the same ACI fabric, different pairs of anchor leaves can be designated for each cluster to distribute peering load and resources effectively.



Centralized Routing Model

ECMP Considerations

- **ACI ECMP Path Limit:** By default, ACI installs up to 16 equal-cost paths for eBGP/iBGP routes. If more than 16 Kubernetes `nodes` are configured for BGP peering, ACI's maximum ECMP path limit can be increased (typically up to 64, check specific hardware/software limits).
- `externalTrafficPolicy: Cluster` **Requirement:** Features like Cilium's Maglev resilient hashing and Direct Server Return (DSR) necessitate setting the `externalTrafficPolicy` to `Cluster` on Kubernetes Services. This setting means every node peering with ACI via BGP advertises itself as a potential next hop for every externally exposed Service, regardless of whether it currently hosts a backend Pod for that Service.
- **Path Selection with > Max ECMP Nodes**: The ACI ECMP selection algorithm installs routes up to the configured maximum number of paths per destination prefix (i.e., per exposed Service `/32`). If more equal-cost paths are available than the configured limit (e.g., 20 nodes peering but max ECMP is 16), ACI uses a hash-based mechanism to select a stable subset of next hops from the available pool. While not all peering nodes might be simultaneously active in the forwarding path for a single service prefix on a single leaf, traffic distribution across the peering nodes remains generally fair system-wide. Furthermore, because DSR ensures the return traffic bypasses the initial ingress node, the potential impact of uneven ingress forwarding is mitigated.

Note: This Advanced Design **requires ACI version 6.1(2) or later.** This prerequisite stems from the dependency on both the *Propagate Next-Hop and Ignore IGP Metric* BGP features, which are essential for optimal routing and path selection in this topology.

© 2025 Cisco and/or its affiliates. All rights reserved.

Cilium Egress Design Options

Regarding the design for handling egress traffic originating from Kubernetes Pods, two primary options can be evaluated based on specific requirements and ACI scale considerations.

Option 1: Egress IP Advertisement via BGP (Preferred)

Cilium can advertise the specific IP addresses used by its Egress Gateway feature directly via BGP. This can be done easily by adding in the `IsovalentBGPAdvertisement` CRD the `advertisementType: EgressGateway` ACI External EPGs (ExtEPGs) can then be used to classify this egress traffic based on the advertised Egress IP prefixes, allowing contracts to be applied.

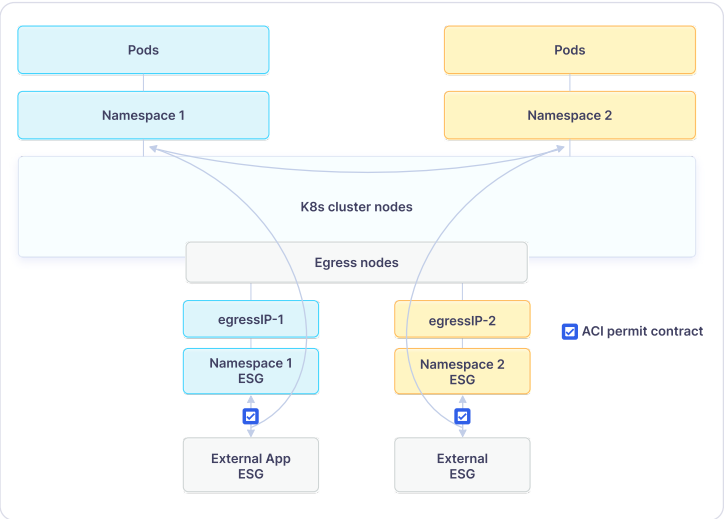
If the number of unique Egress IPs leads to concerns about ACI ExtEPG scale limits, *and/or* if egress traffic inspection via a firewall is required anyway, an alternative approach involves using a single ExtEPG that matches the entire Egress Gateway IP subnet. Service Graph redirection can then be employed to forward all traffic matching this ExtEPG to the firewall for policy enforcement.

This BGP-based option maintains design simplicity, as all nodes (including those potentially acting as egress gateways) can remain topologically identical, connecting primarily through the main L3Out.

Option 2: Egress IP Classification with ESGs

Alternatively, the capabilities of ACI Endpoint Security Groups (ESGs) can be harnessed to create a distinct structure for managing egress traffic:

- **Dedicated Egress Network Segment:** Nodes designated to perform egress functions are configured with a secondary network interface. This interface connects to a separate ACI Bridge Domain (BD) and EPG, distinct from the main L3Out used for node-to-node and service traffic.
- **ESG Classification within Egress EPG:** ESGs are defined within this dedicated egress EPG. IP-based selectors within these ESGs are used to classify traffic based on the specific, static `Egress IP` addresses assigned by the Cilium Egress Gateway feature.
- **Cilium Egress Gateway Policies:** Standard Cilium Egress Gateway policies are implemented within Kubernetes to associate specific Pods or Namespaces with designated egress nodes and their corresponding fixed egress IP addresses. This ensures predictable source IPs for outbound traffic from different application groups.
- **Policy Enforcement via ESGs:** By mapping specific Egress IPs to distinct ESGs, ACI contracts can be applied between these ESGs and external destinations (represented typically by ExtEPGs or other ESGs), enabling granular control over outbound traffic flows at a Namespace or application level directly within ACI policy.



Egress Gateway Traffic Flow using ESGs

It is important to emphasize that this design specifically targets traffic *leaving* the cluster via the designated egress interfaces. Internal cluster traffic (pod-to-pod) remains unaffected by these ESG configurations and continues to flow over the primary node network.

Egress Node Requirements (for ESG Option)

When implementing the ESG-based egress design (Option 2), the designated Egress nodes require configuration with two network interfaces:

© 2025 Cisco and/or its affiliates. All rights reserved.

- **Primary Node Interface:** Connects to the ACI L3Out (shared with other nodes). This L3out handles node-to-node communications and potentially Kubernetes service traffic if the node also participates in BGP. Its IP address should be configured as the primary `node-ip` for kubelet.
  - It is not strictly necessary for nodes dedicated solely to egress traffic to establish BGP peering via this interface, although they can if also serving ingress traffic.
- **Dedicated Egress Interface:** Connects to the separate ACI BD and EPG designated for egress traffic, where ESGs are configured. This interface carries the actual Pod-initiated egress traffic sourced from the `Egress IP` addresses.

Note: For nodes configured with multiple interfaces, it is fundamental to ensure that kubelet is configured to use the correct primary `node-ip` —in this design, that must be the IP address of the interface connected to the main ACI L3Out. Cilium relies on this primary node IP for its internal pod-to-pod east-west routing decisions and typically does not provide mechanisms to selectively route pod traffic over alternative interfaces based on destination.

Routing Considerations (for ESG Option)

A potential challenge with the dual-interface egress node setup (Option 2) is ensuring symmetric routing. Traffic initiated by Pods via the dedicated egress interface (using an `Egress IP` as the source) must receive replies back through the same egress interface. If return traffic mistakenly attempts to exit via the primary node interface (connected to the L3Out), it might be dropped due to RPF checks or firewall rules.

To enforce symmetric routing for egress flows, Linux Policy-Based Routing (PBR) is typically configured on the egress nodes:

- **Create a separate routing table:** For example, table ID `100`.
- **Add default route in the new table:** In table `100`, configure a default route pointing to the ACI gateway IP address residing on the dedicated egress BD/subnet.
- **Create IP routing rules:** Use `ip rule add` commands to direct traffic sourced from the specific `Egress IP` addresses assigned to this node to use routing table `100`.

This ensures that any traffic originating from the node's `Egress IP`(s) uses the dedicated egress path and its associated gateway, preserving routing symmetry for the duration of the connection.

General Egress Node Deployment Considerations:

Regardless of the chosen egress design (Option 1 or 2), careful consideration should be given to the number and placement of `egress nodes`:

- **Redundancy:** Deploy a minimum of two `egress nodes`, preferably connected to different ACI leaf switches (or leaf pairs) to provide resilience against node or leaf failures and facilitate hitless upgrades.
- **Dedicated vs. Shared:** Depending on cluster scale, egress traffic volume, and policy requirements, dedicating specific nodes solely for the egress function might be beneficial. This isolates egress workloads and simplifies resource management compared to having nodes perform ingress, egress, and regular workload functions simultaneously.

Note: A single egress node can be configured with multiple `Egress IP` addresses. This allows one node to serve as the egress point for several distinct egress identities (e.g., different Namespaces or application tiers), improving resource utilization. For instance, Egress IP-A could be mapped via Cilium policy to Namespace A, while Egress IP-B on the same node is mapped to Namespace B.

© 2025 Cisco and/or its affiliates. All rights reserved.

Design Trade-offs

This Advanced Design aims to provide an easily managed and highly scalable solution; however, it presents certain trade-offs compared to simpler alternatives:

1. `externalTrafficPolicy: Cluster` **Requirement:** External Kubernetes Services must use `externalTrafficPolicy: Cluster` due to the reliance on Cilium's Maglev hashing and DSR. This means BGP advertises all peering nodes as potential next hops for every service. The potential downsides (like traffic hitting a node without a local backend Pod) are largely mitigated by DSR ensuring optimal return paths.
2. **Potential Egress Bottlenecks:** Concentrating egress traffic through a potentially small set of dedicated egress nodes (especially in Option 2) could create performance bottlenecks if not sized appropriately.
3. **Reduced Node IP Visibility in L3Out:** Nodes primarily interact with ACI via the Floating SVI L3Out. While ExtEPGs can classify nodes, direct visibility and policy based on individual node IPs within ACI might be less straightforward compared to the Basic Design where nodes might be in direct EPGs.
4. **Increased Complexity (Egress Option 2):** If the ESG-based egress approach (Option 2) is chosen, the configuration becomes more complex due to the need for secondary interfaces, separate BD/EPG/ESGs, and Policy-Based Routing on the egress nodes.

Issue (1) is an inherent aspect of using Maglev/DSR for optimal load balancing and resilience. Issue (2) can generally be addressed through appropriate horizontal scaling (adding more egress nodes) or vertical scaling (using more powerful nodes). Issue (4) is specific to the chosen egress method.

ACI BGP Design

Regardless of whether the Basic Design or the Advanced Design is used, the ACI BGP configuration remains largely the same.

The goal of the ACI fabric in this context is to provide reliable transport and security for the Kubernetes infrastructure. As covered in the ACI Designs section, ACI requires configuration with a Bridge Domain and L3Out to establish connectivity to the clusters.

L3Out Physical Connectivity

There is no strict requirement for the physical connectivity of the L3Out, provided it meets the necessary redundancy level. Implementing L2 redundancy can improve failover times by eliminating the need to wait for BGP convergence.

This design utilizes the floating SVI feature for L3 connectivity. This feature enables the configuration of an L3Out without specifying logical interfaces, thereby removing the need to configure multiple L3Out logical interfaces to maintain routing when VMs migrate between hosts. Floating SVI is supported for VMware vSphere Distributed Switch (VDS) starting from Cisco ACI Release 4.2(1) and on physical domains starting from Cisco ACI Release 5.0(1). Using physical domains is recommended for the following reasons:

- Support for any hypervisor.
- Support for mixed-mode clusters (VMs and bare-metal).

This recommendation is not a strict requirement. If all nodes operate within a VMware environment, there are no technical impediments to using VMM integration. Employing floating SVI also relaxes the constraint against Layer-2 communications between the nodes, enabling the design to utilize:

- A single subnet for all `nodes`.
- A single encapsulation (VLAN) for all `nodes`.

© 2025 Cisco and/or its affiliates. All rights reserved.

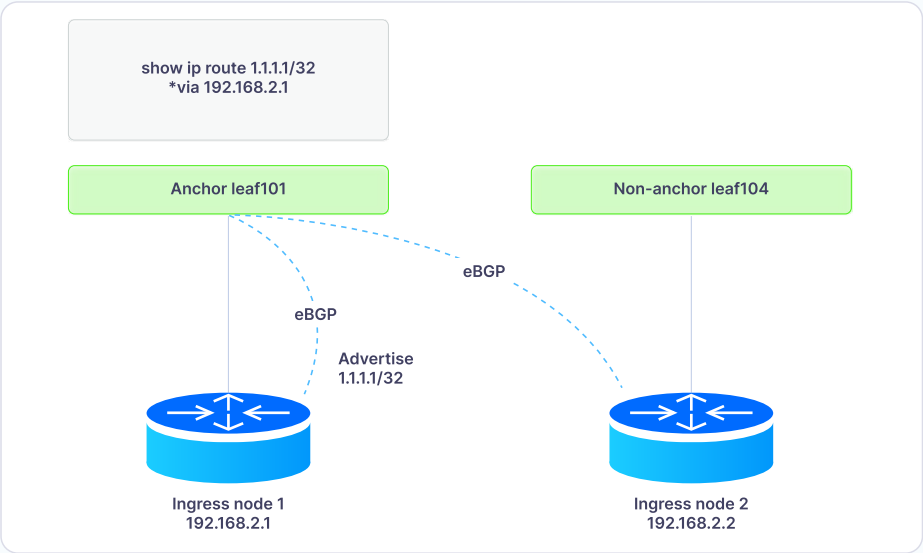
BGP Dynamic Neighbors

All `nodes` configured for BGP peering will share the same AS number. Sharing the same AS number allows the design to leverage the BGP Dynamic Neighbors capability. This feature facilitates the dynamic and automated establishment of peering sessions between routers. Instead of mandating manual configuration of individual neighbor statements for each peer, this feature employs a `listener` (the ACI fabric in this context) to accept incoming BGP connections from a predefined IP subnet.

BGP Peering with ACI Prior to 6.1(2)

In ACI software releases before 6.1(2), a specific peering configuration is necessary: `ingress nodes` must be peered either exclusively with directly attached anchor nodes or exclusively with non-directly attached anchor nodes.

For example, if a service with the IP address `1.1.1.1` is exposed via BGP, an anchor leaf like `leaf101` will only install one ECMP path for `1.1.1.1/32` through `192.168.2.1` if that path originates from a directly connected node. The locally attached route is preferred, even if multiple nodes could potentially provide the service.



Pre-ACI 6.1(2) limitation

This limitation was removed in ACI release 6.1(2). Therefore, using ACI 6.1(2) or a newer version is strongly recommended.

Next-Hop Propagation and Ignore IGP Metric - ACI 6.1(2)+ Required

Warning: These two features are recommended for the Basic Design and are a **requirement** for the *Advanced Design*.

When an external router is physically connected to a non-anchor leaf node, traffic originating from a Cisco ACI internal endpoint destined for an external location typically routes through an anchor leaf node before reaching the external router via the non-anchor leaf. This results in a suboptimal traffic path. Furthermore, as previously noted, the preference for routes from nodes directly connected to anchor leaves can lead to the underutilization of nodes not directly connected.

Enabling the following two features addresses both of these issues:

- **Next-Hop Propagation:** This configuration is applied exclusively to the anchor leaf nodes. It allows them to redistribute external prefixes within the Cisco ACI fabric, using the next-hop IP address of the external router that announced these prefixes. Consequently, compute leaf nodes receive and install these external prefixes in their forwarding tables with the external router's IP address (the Cilium Ingress Gateway in this design) as the next-hop.
- **Ignore IGP Metric:** Configuring this option modifies the BGP best path algorithm to disregard the IGP metric associated with the next hop. This change allows nodes connected to both anchor and non-anchor leaves to be selected as valid next hops, ensuring that all Kubernetes (K8s) nodes are effectively utilized.

© 2025 Cisco and/or its affiliates. All rights reserved.

It is important to note that the Next-Hop Propagation feature was introduced in ACI release 5.0(1). However, support for the Ignore IGP Metric feature was not available until ACI 6.1(2). For this design to function optimally, both features are required. Configuring only Next-Hop Propagation is insufficient. If the environment runs ACI 6.1(1) or an earlier version, the recommendation is to connect the `ingress nodes` directly to the `ACI Anchor nodes`.

Max BGP ECMP Path (Optional)

By default, ACI installs a maximum of 16 eBGP/iBGP Equal-Cost Multi-Path (ECMP) paths. If the deployment requires more than 16 `nodes`, ACI can be configured to support up to 64 ECMP paths.

BGP Graceful Restart

Both ACI and Cilium should be configured to utilize BGP Graceful Restart. This feature ensures that when a BGP speaker restarts its BGP process or if the process unexpectedly terminates, its neighbors retain the previously received paths. This mechanism maintains connectivity without disruption, provided the data plane remains correctly programmed during the restart.

BFD with Cilium BGP

Bidirectional Forwarding Detection (BFD) is a network protocol designed to detect faults along the path between two forwarding engines, like routers. It is frequently used alongside routing protocols such as BGP (Border Gateway Protocol). BFD complements BGP operations by offering rapid and dependable detection of link failures, which facilitates faster failover and enhances overall network resilience. Leveraging this capability makes it possible to achieve sub-second failure detection for the BGP peering sessions.

Warning: If BFD is configured for sub-second failure detection and virtual machines are in use, a live migration event might cause BFD to incorrectly perceive the migrating node as down. This can trigger a BGP session reset, resulting in the withdrawal of all routes previously advertised by that node from the routing table.

BGP Hardening (Optional)

To safeguard the ACI system against potential BGP misconfigurations originating from the Kubernetes environment, implementing the following hardening measures is recommended:

- Enable BGP password authentication between peers.
- Set the maximum accepted AS-path length limit to one.
  - Based on the eBGP design described, the expected AS path length should consistently be one.
- Configure BGP import route control policies to accept only the anticipated external service subnets.
- (Optional) Impose a limit on the maximum number of prefixes accepted from the `nodes`.

© 2025 Cisco and/or its affiliates. All rights reserved.

ACI Scale Testing

The ACI Advanced Design detailed within this document has undergone scalability testing aimed at verifying its effectiveness and reliability across diverse conditions. However, it is crucial to recognize that these tests were performed based on generalized scenarios and assumptions. Given that each organization possesses unique requirements and architectural nuances, conducting independent testing is strongly recommended to validate the design's performance and suitability within the specific deployment environment.

Customized testing facilitates the identification of potential issues stemming from unique architectural components or specific use cases relevant to the organization. This validation process helps ensure that the proposed solution meets performance expectations and integrates seamlessly with existing systems.

The Advanced design has currently been tested under the following conditions:

- A 350-node Kubernetes Cluster.
- Each node advertising 250 BGP `/32` Service Routes.
- All nodes peering with 2 ACI Border Leaves.
- An ACI Fabric composed of 4 leaf switches.
- Clients accessing services via:
  - An L3Out.
  - An EPG/ESG.

Generic ACI Scale Considerations

Within the context of the Cilium and Cisco ACI integration design, the following ACI scale limits should be considered:

Note: These metrics correspond to ACI version 6.1(2). If a different ACI version is deployed, consult the official Verified Scalability Guide for that specific version, as limits may vary. The current date is May 2, 2025.

- **Floating L3Out**: Maximum of 6 anchor leaves and 32 non-anchor leaves.
- **IPs per MAC:** 4096.
- **BFD Neighbors:** 2,000 sessions per leaf using minimum BFD timers: minTx: 300ms, minRx: 300ms, multiplier: 3.
- **BGP Neighbors:** Up to 2,000 per leaf. Fabric-wide scale depends on prefix count and path diversity (e.g., ~70,000 external prefixes with a single path per neighbor might reduce the total neighbor count below the theoretical 20k fabric maximum).
- **Shared L3Out (Inter-VRF Leaking):** 2,000 IPv4 prefixes per L3Out instance used for leaking.
- **External EPGs per L3Out:** 250 per L3Out instance, with a fabric-wide limit of 600 ExtEPGs across all L3Outs.
- **ESGs per Fabric:** 10,000.
- **ESGs per VRF:** 4,000.
- **ESGs per Tenant:** 4,000.
- **L3 IP Selectors per Leaf:** 5,000.
- **IP Longest Prefix Match (LPM) Entries:** Approximately 20,000 IPv4 entries with the default dual-stack hardware profile. Note that altering the hardware profile can impact other scale limits, including the number of supported ECMP paths.

© 2025 Cisco and/or its affiliates. All rights reserved.

Conducted Tests:

Adding/Removing BGP Peers

Test Scenario: A Kubernetes node's participation in BGP peering was toggled by removing and subsequently reapplying the specific Kubernetes label that enables Cilium BGP on that node.

Observed Impact: No discernible traffic impact was observed. This behavior is expected due to the nature of Kubernetes service routing (often using mechanisms like Maglev hashing for backend selection). Even if a node is temporarily removed from BGP peering but continues to receive traffic destined for a service IP, Kube-proxy or Cilium's eBPF datapath on that node can typically still forward the traffic directly to an appropriate backend Pod running locally or tunnel/route it to another node hosting a valid backend Pod.

Reloading a Kubernetes Node

Test Scenario: A Kubernetes node participating in BGP peering was gracefully reloaded.

Observed Impact: Minimal traffic disruption was observed. Potential packet drops can occur during the BGP routing table reconvergence period – specifically, if traffic is forwarded to the reloading node *after* it has started shutting down but before ACI (or other peers) have removed it as a valid next hop via BFD/BGP updates. This potential impact can be further minimized by preemptively removing the node from the BGP peering configuration (e.g., by removing the relevant label) before initiating the reload, allowing routes to withdraw gracefully.

Cilium Agent Upgrade/Restart

The BFD and BGP processes related to external peering typically run within the Cilium agent pod on each Kubernetes node. Restarting the Cilium agent pod (e.g., during a Cilium upgrade or for other maintenance reasons) will cause the BFD session with ACI to drop. However, due to the BGP Graceful Restart capability configured on both ACI and Cilium, the overall impact is minimized.

Test Scenario: The Cilium agent pod on a node participating in BGP peering was restarted.

Observed Impact: Minimal to none. Although the BFD session drops, the ACI leaf maintains the BGP routes learned from that node due to BGP Graceful Restart being triggered. Forwarding continues using the existing routes while the Cilium agent restarts and re-establishes the BGP/BFD sessions.

Reloading an ACI Anchor Node

Test Scenario: An ACI anchor leaf switch, participating in BGP peering with Kubernetes nodes, was reloaded non-gracefully (simulating a crash or power loss, where BFD "down" messages might not be sent reliably before failure).

Observed Impact: Minimal impact, primarily limited to potential loss of in-flight packets traversing the specific anchor leaf at the moment of failure. The overall routing stability for Kubernetes services remained high. Because the design utilizes Next-Hop Propagation (when running ACI 6.1(2)+ and configured appropriately), the ultimate next-hop IP address for the `/32` service routes installed on other ACI leaves is the IP address of the originating Kubernetes node, not the anchor leaf itself. Therefore, the failure of a single anchor leaf does not invalidate the primary routes used by other leaves to reach the services via the Kubernetes nodes. Traffic simply avoids the failed anchor and utilizes paths through remaining anchor/non-anchor leaves connected to the Kubernetes nodes.

© 2025 Cisco and/or its affiliates. All rights reserved.

Example configuration – ACI Advanced Design with OpenShift

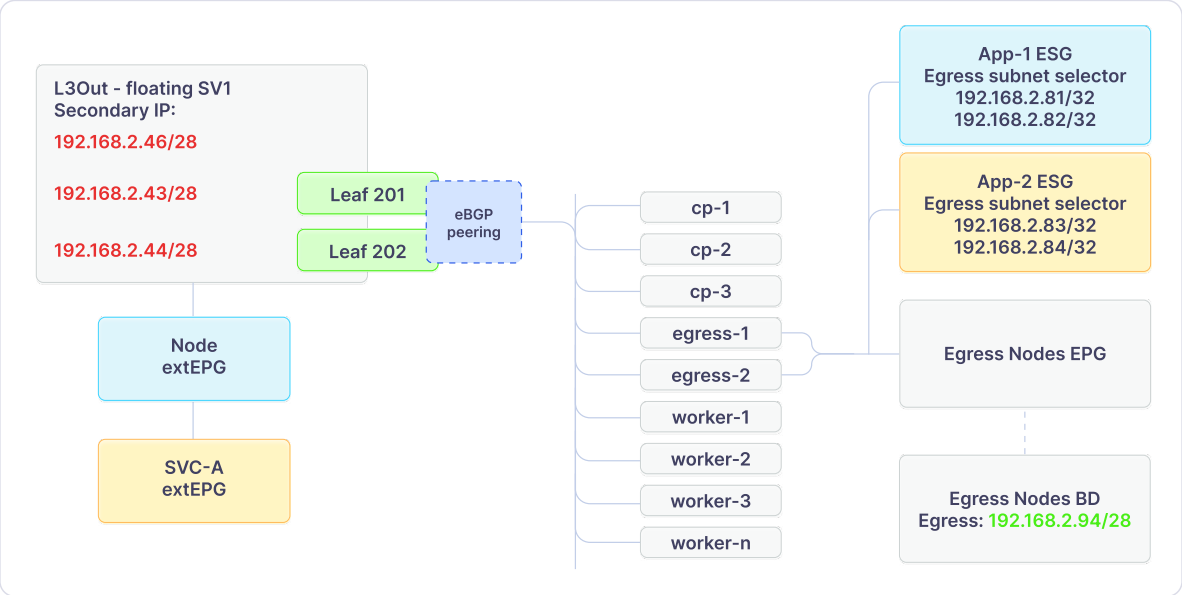
This cluster will be composed of:

- 3x Control Plane nodes
- 3x Worker nodes
- 2x Egress nodes

The following subnets will be used:

- Node subnet and BGP Peering: `192.168.2.32/28`
- Egress Node subnet: `192.168.2.80/28`
- Load-balancer services pool: `192.168.2.48/28`

Note: In this example the ESG configuration is optional and can be completely removed if you choose to advertise the Egress IP over BGP as explained in the Cilium Egress design section.



ACI configuration

ACI configuration

- Create a new `ocp-cluster-1` tenant. All of the subsequent config is placed inside this tenant.
- Create a new `egress` BD.
  - **Note:** In this example, this BD is mapped to a pre-existing VRF in the common tenant. Alternatively, you can create a VRF within the new tenant.
  - Add the `egress` nodes subnet SVI `192.168.2.94/28` as the subnet to the BD.
- Create a new `k8s` application profile.
- Create an `egress` EPG and provision connectivity to your K8s nodes through VMM, static port binding, or the AAEP (Attachable Access Entity Profile).
- Create an `App-1` ESG and configure `192.168.2.81/32` and `192.168.2.82/32` as IP selectors for the ESG.
  - **Note:** This setup makes use of two dedicated egress nodes:
    - `egress-1` is configured with `IP 192.168.2.81/32`

© 2025 Cisco and/or its affiliates. All rights reserved.

    - `egress-2` is configured with `IP 192.168.2.82/32`
  - Add any required contracts to allow for communication such as reaching the cluster.
- Create a new L3Out called `openshift`. This will be configured with a floating SVI.
  - Create a new `Node Profile` and add the anchor leaf nodes to it. This example makes use of two anchor nodes.
  - Configure a floating interface profile with 2 floating SVI interfaces, one for each leaf node.
  - Add a BGP peer connectivity profile for the `192.168.2.32/28` subnet. Select a BGP remote AS for your cluster. This example makes use of AS number 65113.
    - Enable `Bidirectional Forwarding Detection`.
    - Repeat this step for all the interface profiles
  - Create a `nodes` external EPG for the load-balancer services pool: `192.168.2.32/28`, and configure this subnet as `External Subnets for External EPG`. This external EPG will be used to control communication to and from the Openshift nodes.
  - (Optional) Create a `default_svc` external EPG for the load-balancer services pool: `192.168.2.48/28`, and configure this subnet with `External Subnets for External EPG`. This could be useful if you want to have a default set of permit/deny rules for the services that are exposed from your Kubernetes Cluster. The following steps describe how to further restrict access per
  - Create an `app1_svc` external EPG for the load-balancer services: `192.168.2.49/28`, and configure the subnet with `External Subnets for External EPG`.
  - Create an `app2_svc` external EPG for the load-balancer services: `192.168.2.50/28`, and configure the subnet with `External Subnets for External EPG`.
    - **Note:** The IP addresses are allocated dynamically by Cilium, even if you are following this example step by step you might need to update these IPs depending on the actual `LoadBalancer` IP allocation.
  - Add the required `contracts` to `external EPGs` to provide connectivity towards your clients.
- Enable Next Hop Propagate and Ignore IGP Metric (Requires ACI 6.1(2) or newer)
  - Create a `BGP Best Path Policy`
    - Enable `Ignore IGP metric when choosing multipaths`.
    - **Note:** This config is applied to the whole VRF and will apply to any other L3Outs in this VRF.
  - Create a `BGP Protocol Profile` by right clicking on the `Logical Node Profile`
    - Select the `BGP Best Path Policy` we just created for the `Bestpath Control Policy`
  - Create a `Match Rule`
    - Create a Match Prefix for the Load-balancer services pool: `192.168.2.48/28` and set Aggregate to `True`
  - Create a `Set Rule`
    - Enable Next `Hop Propagation`
  - Create a `Route Maps for Route Control`
    - Create a `Context`
    - Add a `Match Rule` and select the `Match Rule` created previously
    - Select as `Set Rule` and select the `Set Rule` created previously
  - Apply the `Route Map` to every `BGP Peer Connectivity Profile` under `Route Control Profile` in the `Import` Direction.

© 2025 Cisco and/or its affiliates. All rights reserved.

- Enable BFD:
  - Create a `BFD Interface Policy` and configure it to match the Cilium Default BFD Timers
    - Detection Multiplier: 3
    - Minimum Transmit Internal: 300
    - Minimum Receive Internal: 300
    - Echo Receive Internal: 50
    - Echo Admin State: Enabled
  - Create a `BFD Interface Profile` under the `Logical Interface Profile` and select the BFD Interface Policy

Warning: If BFD is configured for sub-second failure detection and you are running virtual machines, a live migration may cause BFD to perceive the node as down. Consequently, the BGP session will be reset, leading to the withdrawal of all routes advertised by the affected node from the routing table.

NX-OS EVPN/VXLAN Design with Cilium

This design option enables interoperability between EVPN/VXLAN-based fabrics running NX-OS and Kubernetes clusters using Cilium. While a fully routed IP fabric typically satisfies connectivity requirements from a Kubernetes perspective, real-world environments often include diverse workloads sharing the same network fabric. For greenfield deployments or networks dedicated solely to Kubernetes, a fully routed IP BGP design can be considered.

The network infrastructure in this design is composed of the following components:

- Nexus EVPN/VXLAN fabric (Standalone or managed by Nexus Dashboard)
- Centralized routing design whereby a pair of anchor nodes provides an eBGP control plane
  - BGP Peering is established with all or a subset of the nodes for EgressIP and Load Balancing (VIP) advertisements
- Security Groups and Contracts to control North/South communication
- An anycast gateway providing a L2 segment for node-to-node communication:
  - All the nodes within a Kubernetes cluster are placed in the same L2VNI with an anycast gateway on the TOR
- Each node is connected to the fabric using a single interface, preferably a bond:
  - This (logical) interface is also used for egress (pod-initiated traffic) through secondary IPs on “egress” nodes

This design gives us the following capabilities:

- Secure the traffic to/from the cluster with Security Groups and Contracts.
- DHCP relay support. This allows the Kubernetes nodes to be bootstrapped without the need to manually configure their IP addresses easing the cluster bootstrap and horizontal scalability
- The nodes can be of any type and can be mixed: it is possible to have a cluster composed of bare-metal hosts and VMs running on any hypervisor as long as network connectivity is provided
- Routing simplicity: the node default gateway is the anycast gateway on the TOR
- No need to advertise the POD subnet (PodCIDR) to the fabric
- BGP based ECMP for External K8s service Load Balancing with Resilient Hashing
- Near optimal traffic flows thanks to Direct Server Return

© 2025 Cisco and/or its affiliates. All rights reserved.

Cluster Physical Connectivity

There is no strict requirement for the physical connectivity of the nodes in the Kubernetes cluster as long as it provides the desired redundancy. Most designs are likely to lean toward a vPC based design. Having L2 redundancy improves failover times as there is no need to wait for BGP convergence. For optimal availability a bond should be configured on each node, connected to a pair of TOR switches providing the anycast gateway functionality.

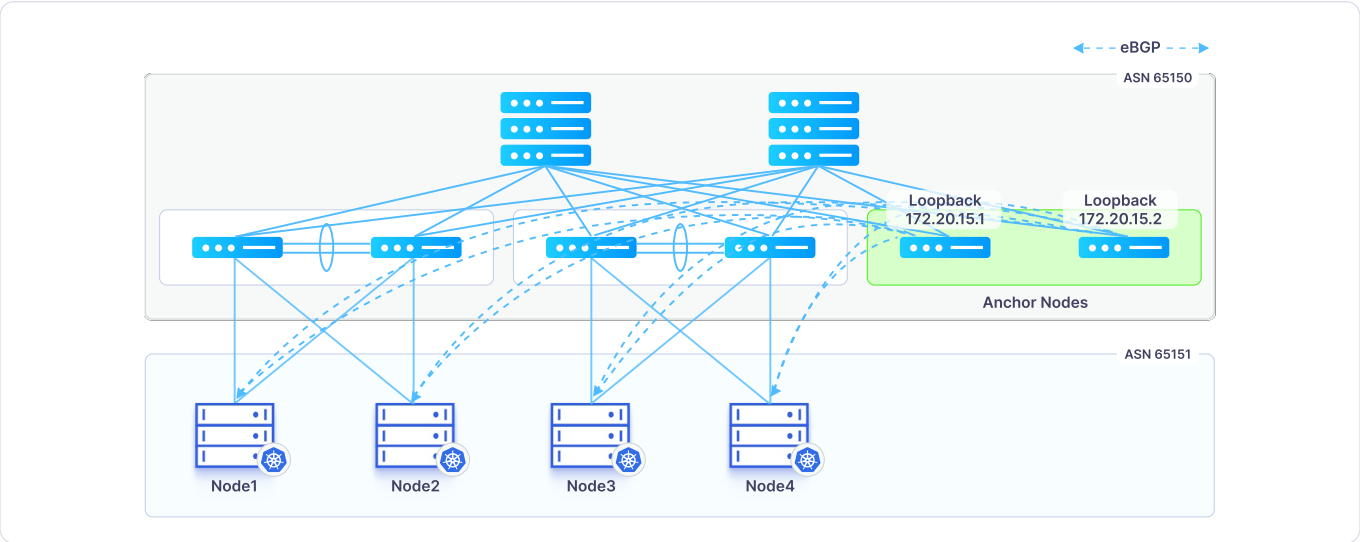
BGP Design

Centralized BGP peering for service advertisement

All Kubernetes nodes will peer with a single pair of switches (also called anchor nodes), irrespective of their physical placement in the datacenter. This simplifies the configuration of the physical network and Cilium. eBGP Peering is established with a loopback interface on each anchor node. The node IPs will be learned in the fabric as type-2 routes.

At the time of writing NX-OS 10.5(3) supports up to 1024 BGP peers per switch. It is unlikely that this will pose a scale issue for a single Kubernetes cluster. In case multiple clusters are connected to the same network fabric, different pairs of anchor switches can be used to distribute the resources.

Each cluster is configured to use a single AS to simplify the configuration.



Centralized Routing

When it comes to the Cilium ingress design, the only consideration is how many `ingress nodes` to deploy and whether to dedicate them only for this purpose.

Ideally, the design should have a minimum of two `ingress nodes` with a redundant connection to a pair of TOR switches. This will provide redundancy in case of `ingress nodes` or TOR failure, and during upgrades.

Depending on the cluster scale and application requirements, dedicated ingress nodes could be beneficial for the following reasons:

- Deterministic latency: if no Pods are running on the `ingress nodes` all ingress traffic will always hit the ingress node first before being routed to the backend pod.
- Limit the number of `ingress nodes`: By having dedicated `ingress nodes`, we can ensure that all the bandwidth and compute are dedicated to the ingress function.
- Reduced number of BGP peerings: As only a subnet of nodes establish BGP peering with the fabric, the management overhead and network state is reduced.
- Specialized hardware: not all the nodes in a Kubernetes clusters need to be the same; specialized high-performance hardware can be utilized for ingress nodes. For example, thanks to CiliumNodeConfig, we can deploy bare-metal nodes with either Mellanox or Intel network interface cards and achieve 100+ Gbps throughput per `ingress node`, thanks to the Cilium Big TCP capability.

© 2025 Cisco and/or its affiliates. All rights reserved.

BGP Dynamic Neighbors

All the `nodes` configured for BGP peering will share the same AS number. By sharing the same AS number, we can use the BGP Dynamic Neighbors capability, which allows for the dynamic and automated establishment of peering sessions between routers. Instead of requiring manual configuration of individual neighbor statements for each peer, this feature utilizes a `listener` to accept incoming BGP connections from a predefined IP subnet.

Example configuration:

```
neighbor 10.58.31.128/28
remote-as 65150
local-as 65151
update-source loopback5
ebgp-multihop 5
address-family ipv4 unicast
```

BGP Graceful Restart

Both NX-OS and Cilium will be configured to use BGP Graceful Restart. When a BGP speaker restarts its BGP process or when the BGP process crashes, neighbors will not discard the received paths from the speaker, ensuring that connectivity is not impacted as long as the data plane is still correctly programmed. This configuration is enabled by default on NX-OS.

BFD With Isovalent Networking for Kubernetes

Bidirectional Forwarding Detection (BFD) is a network protocol used to detect faults in the path between two forwarding engines, such as routers, and is commonly used in conjunction with routing protocols like BGP (Border Gateway Protocol). BFD enhances BGP operations by providing quick and reliable detection of link failures, enabling faster failover and improved network resilience. Thanks to this capability we can achieve sub-second failure detection for the BGP Peering.

Warning: If BFD is configured for sub-second failure detection and the Kubernetes nodes in the cluster are virtual machines, a live migration may cause BFD to perceive the node as down. Consequently, the BGP session will be reset, leading to the withdrawal of all routes advertised by the affected node from the routing table. Note that use of the echo function for BFD is not supported with multi-hop BFD.

Example BFD configuration on the anchor node:

```
router bgp 65101
  vrf ocp_vrf
    neighbor 10.58.31.128/28
      bfd multihop
      bfd multihop interval 300 min_rx 300 multiplier 10
```

BGP hardening (optional)

To protect the fabric against potential Kubernetes BGP misconfigurations, the following settings are recommended:

- Enable BGP password authentication
- Set the maximum AS limit to one
- Configure BGP import route control to accept only the expected external-services and egress subnets.
- (Optional) Set a limit on the number of received prefixes from the nodes.

ECMP Considerations

- The anchor switches will install up to 16 eBGP/iBGP ECMP paths. If more than 16 `nodes` are peering via BGP, the switches can be configured with a higher number of ECMP paths with `maximum-paths n` under the BGP process. This allows the number of ECMP objects to scale to 128. Always refer to the verified scalability guide for the accurate scale number per platform and software version.

© 2025 Cisco and/or its affiliates. All rights reserved.

- To advertise the available ECMP paths from the anchor nodes, and to receive them on other fabric nodes, `additional-paths` and `maximum-paths ibgp #N` should be added to the l2vpn evpn address family under the BGP process.
- Maglev and DSR require the `externalTrafficPolicy` for Services to be set to `Cluster`: This means that every node that peers with the anchor nodes over BGP will advertise itself as a valid next hop for every exposed Service in the cluster. Upon receiving the traffic the node will route the traffic to an appropriate backend using the Maglev algorithm.
- The ECMP selection algorithm will install up to the configured number of ECMP per exposed service. After the ECMP objects are exhausted, there is a fallback to the adjacency for all further routes. This means that even if not all the nodes can be used for the service, traffic is still distributed fairly to all nodes running BGP. Furthermore, since DSR is used, the response is sent back to the client directly from the pod, bypassing the original node that received the request.
- To allow for weighted distribution of traffic to nodes, proportional multipathing should be configured. For more information refer to the following configuration guide: Configuring Proportional Multipath for VNF

Example configuration for one of the anchor nodes for 16 ECMP paths:

```
router bgp 65101
  router-id 172.20.10.7
  address-family l2vpn evpn
    maximum-paths ibgp 16
    additional-paths send
    additional-paths receive
    additional-paths selection route-map match-all
  vrf ocp_vrf
    address-family ipv4 unicast
      advertise l2vpn evpn
      redistribute direct route-map fabric-rmap-redist-subnet
      maximum-paths 16
      maximum-paths ibgp 16
      export-gateway-ip
    neighbor 10.58.31.128/28
      remote-as 65150
      local-as 65151 no-prepend replace-as
      update-source loopback5
      ebgp-multihop 5
      address-family ipv4 unicast

route-map match-all permit 10
  set path-selection all advertise
```

Note that the receiving switches also need to have the `additional-path` configuration to receive the prefixes.

Optional: to select specific prefixes for which to selectively export the gateway IP, configure a route-map which applies `set evpn gateway-ip use-next-hop` to an IP prefix-list that matches the LoadBalancer IP pool.

Example configuration for selective export-gateway-ip:

```
route-map name permit sequence
  match ip address prefix-list name
  set evpn gateway-ip use-next-hop

vrf context vrf
  address-family ipv4 unicast
    export map name
```

© 2025 Cisco and/or its affiliates. All rights reserved.

Here is output from a fabric node that received 6 ECMP paths for a service advertised from 6 Kubernetes nodes. Note that the next hop is set to the primary interface IP of the Kubernetes node.

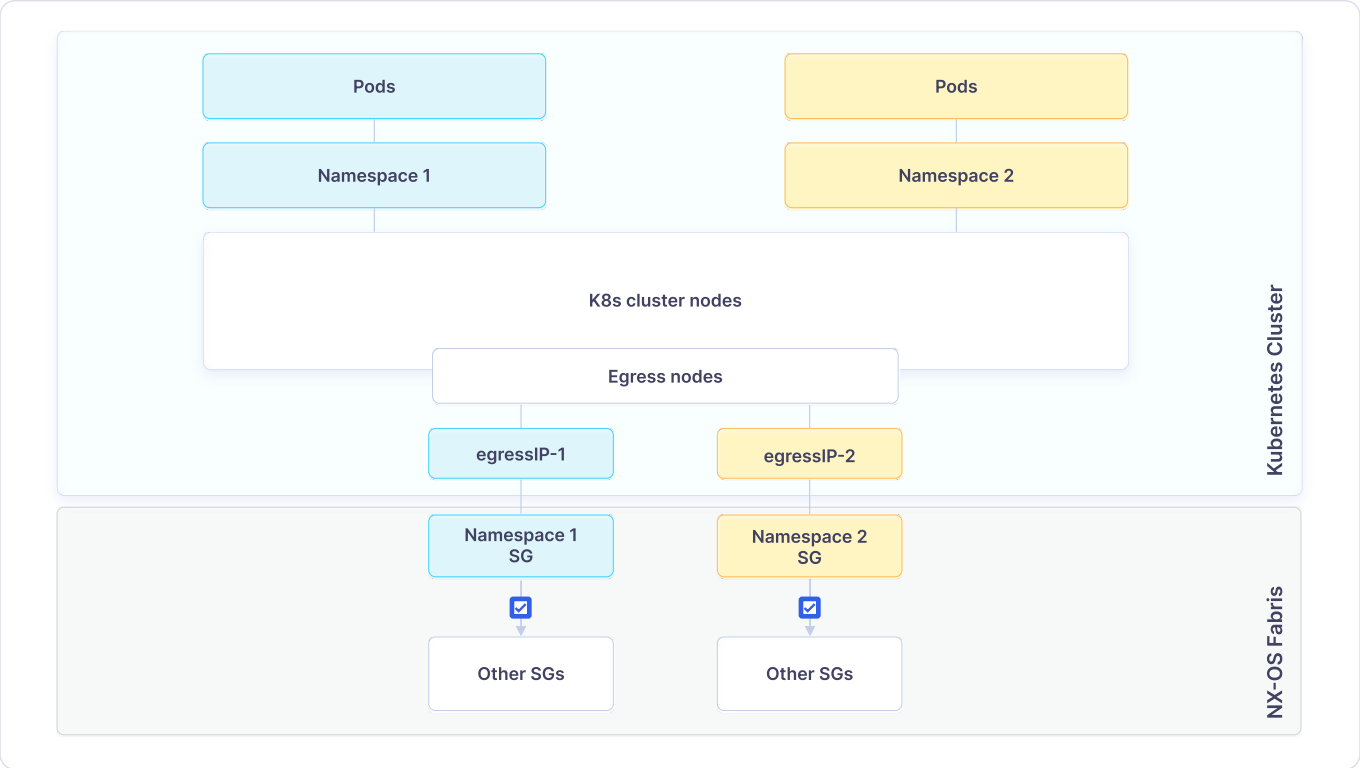
```
bgw-1104(config-router)# show ip route 10.58.31.137 vrf ocp_vrf
IP Route Table for VRF "ocp_vrf"
'*' denotes best ucast next-hop
'**' denotes best mcast next-hop
'[x/y]' denotes [preference/metric]
'%<string>' in via output denotes VRF <string>

10.58.31.137/32, ubest/mbest: 6/0
    *via 10.58.31.129, [200/0], 2d01h, bgp-65101, internal, tag 65150
    *via 10.58.31.130, [200/0], 2d01h, bgp-65101, internal, tag 65150
    *via 10.58.31.131, [200/0], 2d01h, bgp-65101, internal, tag 65150
    *via 10.58.31.132, [200/0], 2d01h, bgp-65101, internal, tag 65150
    *via 10.58.31.133, [200/0], 2d01h, bgp-65101, internal, tag 65150
    *via 10.58.31.134, [200/0], 2d01h, bgp-65101, internal, tag 65150
```

Isovalent Networking for Kubernetes Egress Design

The goal for egress traffic is to make it deterministic by assigning IP addresses to "egress gateway" nodes. These IPs can then be matched using security-groups on NX-OS, so that ACLs can be applied against this traffic. Selectors can be used on Kubernetes to apply an egress policy to a set of pods and or namespaces.

Note that not every node in the cluster needs to assume the `egressGateway` function, which allows for hardware customization and physical placement based on egress requirements.



Egress Gateway traffic flows

© 2025 Cisco and/or its affiliates. All rights reserved.

- Dedicated Security-Groups for Egress Gateway Traffic: The nodes performing egress should be configured with an additional IP address for security-group matching. This IP can either be a secondary IP on the primary interface, or a secondary interface.
- Cilium Egress Gateway Policies: Implement Cilium Egress Gateway policies to associate specific namespaces with designated gateway nodes, each with a fixed egress IP address. This mapping ensures consistent and predictable IP addresses for pod-initiated traffic that is leaving the cluster.
- Security-Group matching on Egress IPs on the fabric nodes. Apply security-groups matching on the egress IPs and make use of Contracts to enhance security and control over outbound traffic.

It is important to note that this design specifically addresses traffic leaving the cluster. Internal cluster and ingress traffic are unaffected by these configurations.

Egress IP advertisement Over BGP (Preferred Option)

Cilium can Advertise the Egress IP over BGP. This can be done by adding `advertisementType: EgressGateway` in the `IsovalentBGPAdvertisement` CRD.

This option allows for a clean design whereby exposing services (ingress), and pod-initiated traffic via egress follow the same logic and configuration.

Each egress node can be configured with multiple IP addresses, enabling it to support multiple “identities” in the Kubernetes. This allows the same node to be re-used. For example, egress IP-A can be associated with Namespace A, while IP-B can be used for egress traffic leaving Namespace B.

Depending on the cluster scale and application requirements, the use of dedicated `egress nodes` could be beneficial for the same reason as discussed in the ingress considerations.

Securing Ingress and Egress Traffic

As both ingress and egress traffic IPs are advertised using BGP, Security Groups can be configured to apply access control in the fabric. Note that the classification for the external subnet should be done on the receiving anchor nodes.

Example configuration to `deny ICMP` to any destination from the `egress-1` namespace:

```
vrf context ocp_vrf
  security enforce tag 13645 default permit

security-group 150 name ns-egress-1
  match external-subnets vrf ocp_vrf ipv4 10.58.31.152/32  # Egress IP 1
  match external-subnets vrf ocp_vrf ipv4 10.58.31.153/32  # Egress IP 2

class-map type security match-any ocp-deny-icmp
  match ip icmp

policy-map type security ns-egress-1-icmp-log
  class ocp-deny-icmp
    deny
    log

vrf context ocp_vrf
security contract source 150 destination any policy ns-egress-1-icmp-log
```

Example configuration to permit HTTP from any destination to an application:

```
security-group 151 name k8s-app-1
  match external-subnets vrf ocp_vrf ipv4 10.58.31.204/32     # Allocated LB VIP
```

© 2025 Cisco and/or its affiliates. All rights reserved.

```
class-map type security match-any k8s-app1-permit-http
  match ip tcp dport 80

policy-map type security k8s-app1-permit-http
class k8s-app1-permit-http
    permit
    log

vrf context ocp_vrf
security contract source any destination 151 policy k8s-app1-permit-http
```

For more information refer to the NX-OS VXLAN GPO Configuration Guide.

Design Trade-offs

This design aims to provide you with an easy and highly scalable design; however, it comes with the following drawbacks:

1. External services can only be exposed with the `externalTrafficPolicy` set to `Cluster`: This requirement is imposed by Maglev. This means that all nodes that are peering, are a valid next-hop for a particular service. The receiving node may have to send the traffic to another node which contains the backend. This drawback is however of minor consequence thanks to Direct Server Return. This requirement also poses additional strain on the number of IPv4 routes learned on the anchor switches.
2. Potential bottlenecks for egress traffic. By default pod-initiated traffic is SNATed to the node IP address which would distribute egress traffic amongst all nodes. This reduces the level of visibility and control that can be applied. By using egressGateway HA, egress traffic is deterministic and will leave the cluster via dedicated egress nodes. This can add additional latency due to the extra number of hops for traffic egressing the cluster. Should this pose a bottleneck, the egress function can be horizontally or vertically scaled.

Closing Remarks

This document provides comprehensive best practices and design guidance for integrating Isovalent Networking for Kubernetes (Cilium) with Cisco ACI and NX-OS datacenter fabrics, detailing various design options, key features like BGP peering, Maglev, and DSR, and scale considerations. For organizations considering these solutions, the recommended next steps involve carefully evaluating the proposed designs against your specific operational requirements and scale, conducting independent testing within your environment to validate performance and integration, and utilizing the provided contact information for tailored design assistance.

© 2025 Cisco and/or its affiliates. All rights reserved.

Take the next step with Isovalent!

isovalent.site/discover-lab: Get started with Isovalent labs in seconds. Explore use cases around Network Observability, Network Policies, Transparent Encryption, Mutual Authentication, or Runtime Security and Enforcement.

isovalent.site/demo: Book your personalized session today and dive into the full potential of the Isovalent Enterprise Platform.

ISOVALENT
now part of CISCO

Isovalent is the company founded by the creators of Cilium and eBPF. We build open source software and enterprise solutions solving networking, security, and observability needs for modern cloud native infrastructure. Google (GKE, Anthos), Amazon (EKS-A), and Microsoft (AKS) have all adopted Cilium to provide networking and security for Kubernetes. Cilium is used by platform engineering teams such as Adobe, Bell Canada, ByteDance, Capital One, Datadog, IKEA, Schuberg Philis, and Sky.

© 2025 Cisco and/or its affiliates. All rights reserved.